Objective

The goal of this exercise is to design and implement a C++ framework for basic polynomial operations. The framework must rely on inheritance, polymorphism, and templates, and expose its functionality to Python using pybind11.

Background

Let

$$p(x) = \sum_{i=0}^{n} a_i x^i, \qquad q(x) = \sum_{i=0}^{n} b_i x^i$$

be two polynomials with coefficients $a_i, b_i \in \mathbb{R}$ and let $\alpha \in \mathbb{R}$ be a scalar.

The following operations are defined:

- **Polynomial addition**: $(p+q)(x) = \sum_{i=0}^{n} (a_i + b_i) x^i$.
- **Scalar multiplication**: $(\alpha p)(x) = \sum_{i=0}^{n} (\alpha a_i) x^i$.
- **Derivative**: $p'(x) = \sum_{i=1}^{n} i a_i x^{i-1}$.
- **Evaluation**: $p(c) = \sum_{i=0}^{n} a_i c^i$, where c is a scalar.

All operations return a polynomial except evaluation, which returns a scalar.

Instructions

You are asked to build a reusable polynomial operations library in C++. The library must allow different operations to be applied to polynomials through a common interface and must support different numeric types using templates.

Tasks

1. **(2 points) Define a templated `Polynomial` class**

   Define a templated class `Polynomial<T>` that:

   - Stores the polynomial coefficients, sorted properly.
   - Provides constructors for specifying the polynomial degree or directly initializing the coefficients.
   - Provides a method `degree()` that returns the degree of the polynomial.
   - **(Optional)** Briefly discuss (with inline comments in the code) which container from the STL is more appropriate to store the coefficients. What if the polynomial is sparse, i.e., several coefficients are zero?

2. **(2 points) Coefficient access and validation**

   - Implement coefficient access using `operator[]`.
   - Ensure const-correctness.
   - Accessing invalid coefficients must throw an exception.
   - Follow RAII principles and ensure memory safety.

3. **(3 points) Polynomial operation hierarchy**

   - Define an abstract base class:

```cpp
template<typename T>
class PolynomialOp {
public:
```

```
    virtual Polynomial<T> apply(const Polynomial<T>& p) const = 0;
    // ...
};
```

- Implement the following derived classes:
  1. `AddOp<T>`: Adds a second polynomial (provided at construction).
  2. `ScaleOp<T>`: Multiplies the polynomial by a scalar (provided at construction).
  3. `DerivativeOp<T>`: Computes the derivative of the polynomial.
- Each derived class must:
  - Override `apply()`.
  - Validate polynomial degrees where required.
  - Throw exceptions on invalid operations.

4. **(2 points) Polynomial evaluation**

   Implement an `EvaluateOp<T>` class that evaluates a polynomial at a given point.

   - Clearly define how the scalar result is returned.
   - The evaluation point must be provided at construction.
   - Handle invalid cases with exceptions where appropriate.
   - **(Optional)** Use Horner's algorithm for evaluation.

5. **(1 point) Testing and polymorphism**

   In `main()`:

   - Demonstrate the use of polymorphism by applying operations through a base-class pointer or reference.
   - Verify that the framework works for at least two different numeric types (e.g., `int`, `double`).

6. **(1 point) Configuration and compilation**

   - Write a `CMakeLists.txt` file to compile your code into a reusable library and a test executable.
   - Include proper C++17 standard requirements and optimization flags.
   - Provide clear build and usage instructions.

7. **(4 points) Python bindings using pybind11**

   - Expose the `Polynomial` class and all operations to Python.
   - Ensure that C++ exceptions are properly translated into Python exceptions.
   - Write a Python script that:
     - Creates polynomials and applies polynomial operations.
     - Computes derivatives and evaluations.
     - Compares the results and performance with NumPy (`numpy.polynomial`) on high-degree polynomials.
     - Briefly comments on any observed differences.

Evaluation criteria

- Correct use of templates, inheritance, and polymorphism.
- Correctness of polynomial operations.
- Proper handling of edge cases and exceptions.
- Memory safety and RAII compliance.
- Clean and usable Python bindings.
- Meaningful comparison with NumPy.
- Quality and clarity of the CMake configuration.
- Code organization and readability.